Esercizio

Scrivere un server che fornisce alcune informazioni sulle date. Il protocollo di comunicazione con il client è il seguente

Client → Server

DaysToNow#date	- chiede il numero di giorni fra oggi e date
DaysBetween#date1#date2	- chiede il numero di giorni fra date1 e date2
SubDays#date#daysNumber	- chiede date diminuita di daysNumber
AddDays#date#daysNumber	- chiede date aumentata di daysNumber
DayOfWeek#date	- chiede il giorno della settimana di date
LeapYear#year	- chiede se year è bisestile
Quit	- comunica l'uscita

Server → Client

Tutti i messaggi del server sono composti aggiungendo al messaggio del client quanto richiesto

DaysToNow#date#daysNumber	- daysNumber numero di giorni fino a oggi
DaysBetween#date1#date2#daysNumber	- daysNumber numero di giorni fra date1 e date2
SubDays#date#numberDays#date2	- date2 la data ottenuta sottraendo daysNumber a date1
AddDays#date1#daysNumber#date2	- date2 la data ottenuta aggiungendo daysNumber a date1
DayOfWeek#date#dayName	- dayName nome del giorno della settimana
LeapYear#year#isLeap	- isLeap true se year è bisestile, false se non bisestile

Tutte le date sono nel formato yyyy-mm-dd

Gestire gli errori di comando sconosciuto, numero di parametri errati, data o numero malformati

Giorni intercorsi fra oggi e data inviata dal client

```
LocalDate date=LocalDate.parse("2022-08-22");
long day=ChronoUnit.DAYS.between(date, LocalDate.now());
```

Oppure
```
Period p=Period.between(date,LocalDate.now());
```
p avrà i metodi get per anni , mesi e giorni che dividono le due date

Giorno della settimana della data inviata dal client
```
String dayOfWeek = date.getDayOfWeek().getDisplayName(TextStyle.FULL, Locale.ITALY)
```

Data ottenuta da una data cui vengono sommati o sottratti dei giorni
```
LocalDate newDate=date.minusDays(123);
LocalDate newDate=date.plusDays(123);
```

Le eccezioni da gestire sono `DateTimeParseException` per le date errate e `NumberFormatException` per i numeri